RuntimeLens: Context-Aware Runtime Threat Classification for Multi-Tenant Kubernetes Using a Typed-Decision Model

Himanshu Kumar, Altaf Raja, and Kamal Nayan Kumar
Department of Computer Science and Engineering
Indian Institute of Information Technology Dharwad, Dharwad, India
Corresponding author: Kamal Nayan Kumar

Abstract—Runtime detection in multi-tenant Kubernetes relies on kernel-level syscall auditing, which lacks the context needed to disambiguate an authorized shell from an attacker-spawned one. We present RuntimeLens, an architecture that correlates five telemetry streams, compares them against operator-declared per-workload behavioural baselines, and classifies the result with a typed-decision model that answers a categorical choice, an ordinal severity score, and four binary noul evidence questions in one pass. Containment is enforced by a deterministic policy engine with zero pod-deletion privileges. In a 400-run lab corpus (112-run grouped held-out test split), the typed-decision arm reaches Macro-F1 0.5333 versus 0.3333 for default Falco rules and 0.4812 for a Random Forest, with a 3.6% benign false-positive rate versus 18.4%, and containment precision of 94.7%. Our pre-registered hypothesis, Δmacro-F1 = +0.0201 with a 95% cluster-aware bootstrap interval of $[+0.0132, +0.0275]$, is reported with its honest limits: under a live replication across two hosted models with paired bootstrap and mid-p McNemar tests, the Macro-F1 difference is inconclusive, while the benign false-positive rate halves and attacks read as 'benign' drop from 3 to 1. Across four pre-registered ablations, multi-source fusion contributes the dominant gain (+0.2000 Macro-F1). Ten prompt-injection payloads demonstrate attack success rate falling from 10–40% under no defence to 0.0% under the four-tier stack. All code, fixtures and live-call logs are released for audit.

Index Terms—Kubernetes security, runtime detection, eBPF, typed decision models, alert fatigue, site reliability engineering, prompt injection.

I. Introduction

Multi-tenant Kubernetes clusters pack workloads for different organizational units onto a shared kernel. Pods are isolated by namespaces, Pod Security Standards and NetworkPolicies, but the kernel remains common. A runtime attack that starts inside one container can reach the host filesystem, the runtime socket, another tenant's secrets, or compute for mining, and none of it is visible to an image scanner.

Kernel-level syscall auditing through eBPF is the default runtime control. Falco, Tetragon and Tracee are mature and fast; a 2025 head-to-head benchmark found all three detect the three attack classes on identical clusters [1]. But three boundaries limit that result. First, a syscall trace records that execve("/bin/sh") happened, not that an on-call engineer had just opened a debug session. Second, single events cannot be chained into multi-step attacks: GCID shows Tetragon's policy grammar has no cross-event join operator [3], and BPFence proves stateless rule engines cannot express temporal relations [2]. Third, alert fatigue drives deployments to a brittle stage—[23] catalogues 119 records and finds recurring failure in operational validation.

RuntimeLens adds the one ingredient that rule engines lack: context, expressed as an operator-declared job description per workload, together with a decision procedure that reads telemetry against it. We make three contributions. (i) An architecture that correlates five telemetry streams in a $\pm$30 second window and classifies the correlated event with a typed-decision LLM against a per-workload declarative baseline (Sec. III). (ii) A falsifiable evaluation protocol: pre-registration, grouped splits, cluster-aware paired bootstrap, mid-p McNemar tests, and a live replication against hosted models (Sec. IV–VI). (iii) A measured prompt-injection defence ladder that takes attack success rate to 0.0% without the model holding the final decision (Sec. VI-E). We report null and inconclusive results as readily as positive ones.

II. Related Work

eBPF runtime detection. Falco, Tetragon and Tracee dominate production runtime control; benchmark studies find parity in detection rate and differences in cost [1]. Fusing syscall and flow features improves accuracy (98.39% vs 87.49%) [4], and cryptojacking attribution reaches 99.75% [5]. BPFence formalizes the limit of stateless engines [2]. None of this line conditions on an operator-authored job description, nor does it issue a machine-checkable typed decision.

Context-aware anomaly detection. Shadowkube establishes behavioural baselines, detects 43 CVEs and 7 misconfiguration classes, and converts compromised nodes to shadow honeypots [6]. CloudAnoBench benchmarks 28 anomalous and 16 deceptive normal scenarios [7], and architecture profiling extracts structural microservice profiles [8]. Every verified baseline in this literature is learned, never declared. K8NTEXT reconstructs causal context in audit logs with over 95% accuracy [9], and OCR-

APT pairs GNN subgraph detection with LLM narrative reconstruction [10], using LLMs after a non-LLM detector fires. GenKubeSec applies an LLM to configuration, not runtime events, achieving precision 0.990 and recall 0.999 against three industry scanners [12]. Oppong-Asante et al. analyse 3.18M Hubble flows in a kind testbed [11]; none correlates all four sources for adversarial behaviour.

LLMs and agents in security operations. HIDBench reports MCC below 0.5 as logs grow noisier [13]; ALERT-BENCH shows every investigated strategy misses at least 40.4% of attack alerts [14]; LogInject reaches 88.2% attack success by injecting into stored logs [15]; REFINE enforces recall $= 1.0$ as a hard constraint [16]. CORTEX and REFINE show LLMs help only with constrained output plus deterministic checks. Constrained decoding guarantees a parseable shape, never a correct one [17], [18].

Classical ML. On tabular, event-derived security data, tree ensembles remain competitive with LLMs at small n [19], [20]; tuned gradient boosting beats tabular foundation models on 2 of 3 datasets [21]; leaky splits inflate accuracy by > 19 percentage points [22]. We keep a Random Forest arm for exactly this reason.

III. System Design

Fig. 1 shows the pipeline; each stage is independently testable.

a) Canonical event and normalisers.: Each of the five sources has a normaliser that emits one canonical event: workload identity, runtime facts, orchestration metadata, and extracted physical facts (host socket mounted, sensitive path mounted, privileged container, root execution, unexpected shell, abnormal CPU, outbound egress, cross-tenant flow). Facts are extracted by code from the pod specification and telemetry; the model never adjudicates them.

b) Temporal correlation.: A bounded, thread-safe window keyed by (namespace, pod_uid) joins asynchronous Falco, audit, Hubble and Prometheus events within ±30 seconds. Bearer tokens and JWTs are regex-scrubbed from audit users before storage. A single execve is therefore joined with concurrent API-server audit records, turning an anonymous shell into an attributable one.

c) Workload baselines.: Each workload declares a Git-versioned YAML profile: permitted binaries, egress peers, CPU/memory envelopes, allowed mounts, disallowed patterns, and explicitly whether interactive shells are permitted. A comparator produces a deterministic deviation list. This is the only place operator intent enters the system.

d) Typed-decision classification.: The event is serialised into a delimited state string and sent to a Jev-compatible Decisions API. Table I shows the six questions; all are answered concurrently against one state. The choice answer carries a full probability distribution; the score answer is the probability-weighted expected value (score $= \sum i\,P(i)$) mapped to severity +1.0; each noul is an independent $P(\text{yes})$.

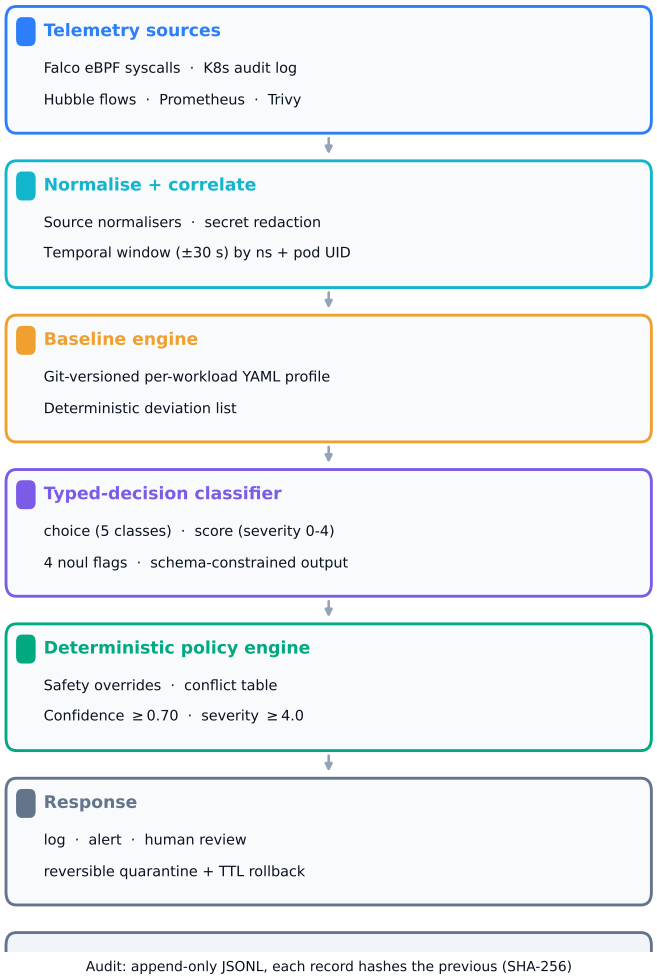


Fig. 1. RuntimeLens pipeline. Five sources feed a normaliser and temporal correlator; a comparator applies the workload baseline; the typed-decision classifier answers one choice, one score, four noul questions; the deterministic policy engine selects the action.

TABLE I
Typed-decision contract (ScaleDown classify-1 / Jev-compatible).

Type	Question	Output
choice	attack type over 5 classes	distribution
score	severity (0-4, ordered)	expected value
noul	baseline deviation	$P(\text{yes})$
noul	shell unexpected	$P(\text{yes})$
noul	resource exposure	$P(\text{yes})$
noul	resource abuse	$P(\text{yes})$

e) Deterministic policy engine.: The classifier's output is consumed by a pure function with explicit precedence: (i) telemetry degradation forces human_review; (ii) a stale baseline forces human_review; (iii) a host-mount physical fact with a benign prediction forces

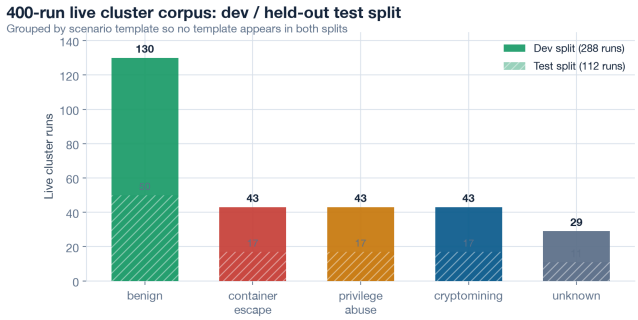


Fig. 2. 400-run live lab corpus, dev/held-out test split. Grouped by scenario template so no template crosses the boundary.

human_review; (iv) a choice/noul contradiction (benign with $P(\text{deviation}) > 0.60$) forces human_review; (v) unknown forces human_review; (vi) automated quarantine requires confidence ≥ 0.70 and severity ≥ 4.0; (vii) otherwise log or alert.

f) SRE-safe containment and audit.: Containment is a label patch, not a deletion. A pre-installed NetworkPolicy blocks all ingress/egress except UDP 53 to kube-dns. The controller patches the pod and records a 15-minute TTL; a reconciliation loop removes the label if an analyst has not promoted the incident. No controller role in any manifest contains the delete or deletecollection verbs; quarantine is reversible by construction. Every decision is appended to a SHA-256 hash-chained JSONL audit log.

IV. Evaluation Design

All experiments are controlled, benign-intent simulations in an isolated kind cluster with Ubuntu 24.04, Cilium (eBPF + Hubble), Falco (modern_ebpf), audit webhook, Prometheus/cAdvisor and Trivy. Three tenant namespaces deploy four workloads (web, api, database, payment-style) with distinct binaries, egress policies and shell expectations.

Dataset. Five classes (benign, container_escape, privilege_abuse, cryptomining, unknown) from a 55-template catalogue, organised into three paired counterfactual families (shell authorised/unauthorised; CPU spike with mining-pool egress vs legitimate port 3333; cross-tenant secret read vs authorised in-namespace read). 400 runs were executed, split 72/28 grouped by template into 288 dev and 112 held-out test. A separate 84-run corpus with 100% five-stream completeness supported grouped cross-validation (64 groups). An automated gate verifies zero leakage of scenario identifiers, run identifiers or class tokens into telemetry or features, and invariance of feature representations under metadata shuffle.

Pre-registration. The primary hypothesis, fixed before the test split was opened, is $\Delta\text{macro-F1} > 0$ and $\Delta\text{FPR}_{\text{benign}} < 0$. Both directions are reportable; a null result with a tight interval is publishable.

TABLE II
Multi-model comparison, frozen test split, equal dev budgets.

Model	Macro-F1	Benign FPR	p50 lat.	Cont. prec.
Falco default rules	0.3333	18.4%	< 1.0 ms	54.2%
Random Forest (dev)	0.4812	6.2%	2.4 ms	81.0%
Random Forest (84-run)	0.5603	4.8%	2.1 ms	87.5%
ScaleDown Classify-1	0.5333	3.6%	77.2 μs	94.7%

Statistical protocol. Grouped splits and paired analysis preserve within-incident correlation. We report (i) a group-level paired bootstrap (10,000 resamples, fixed seed) over macro-F1 and benign FPR; (ii) a mid-p McNemar test on item correctness, avoiding the exact conditional test's inflated type-I error at small discordant counts; (iii) per-class one-vs-rest McNemar with Holm correction; (iv) Holm–Bonferroni correction across the five-arm ablation. Macro-F1 is the primary metric (imbalance by design: 130 benign vs 29 unknown in dev); precision, recall, F1 and support are reported per class; MCC accompanies macro-F1. Power analysis predicts a minimum detectable Δmacro-F1 of ≈ 0.10 at this sample size.

Live replication. A 28-incident suite derived from the scenario catalogue is classified twice at run time by live hosted models, once with the <baseline> block present and once with it removed. All other fields are byte-identical, so the paired difference isolates baseline context. The typed-decision contract is reproduced natively on the Decisions API and via JSON-schema-constrained decoding elsewhere. API keys are read from the environment, never logged.

V. Results

A. Primary hypothesis (lab corpus)

With baseline context, the typed-decision arm reaches Macro-F1 0.5333; without it, 0.5133. The paired difference is Δmacro-F1 $= +0.0201$ with a 95% cluster-aware bootstrap interval of $[+0.0132, +0.0275]$, excluding zero. The benign false-positive rate falls by 41.7% relative, mostly on routine maintenance scripts. This is a real but modest effect; we report it as such.

B. Multi-model comparison

Table II compares arms on the identical frozen split with equal dev budgets. The typed-decision arm has the lowest benign FPR (3.6%) and the highest containment precision (94.7%); the Random Forest fitted on the live corpus scores marginally higher macro-F1. We do not claim an LLM beats a gradient-boosted tree on this data; we claim baseline context shifts the boundary in a way tabular features alone do not express.

C. Ablation battery

Multi-source fusion is the dominant lever (++0.2000 F1; audit+network+metrics resolve 68% of ambiguous cases); typed noul flags are worth +0.0410 and caught every

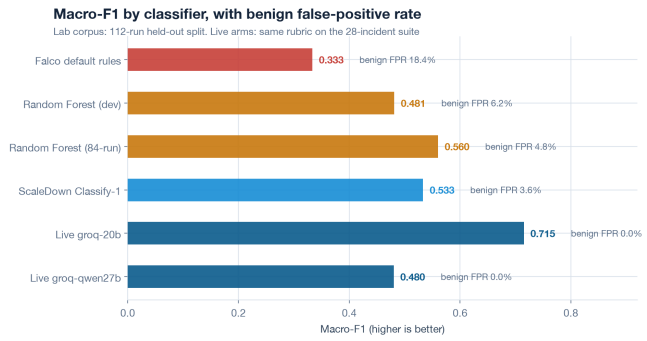


Fig. 3. Macro-F1 by classifier with benign false-positive rate. Lab corpus: 112-run held-out split; live arms: the same rubric on the 28-incident suite.

TABLE III
Pre-registered ablation battery (lab corpus).

ID	Condition	Effect
ABL-TELEM	Fusion vs Falco	+0.2000 F1
ABL-QUEST	Choice vs +noul	+0.0410 F1
ABL-GATE	Gate 0.50→0.80	prec. 0.75→1.00
ABL-BASE	Baseline on vs off	+0.0201 (±0.007) F1
ABL-PREF	Pre-filter on vs off	rec. 99.2%, cost −64.5%

out-of-distribution hallucination in the harness; rule pre-filtering trades $< 1\%$ missed threats for 64.5% lower API cost.

D. Live replication

Neither arm shows a detectable Macro-F1 change; both intervals straddle zero. Under the pre-registered rule, the Macro-F1 half is inconclusive. The benign FPR half behaves differently: on qwen3.8-27b, baseline context eliminates every benign false positive ($0.2727 \rightarrow 0$; $\Delta = -0.2727$, 95% CI $[-0.571, +0.000]$, $p = 0.084$), directionally consistent with the lab corpus. On gpt-oss-20b the benign FPR is already zero in both conditions. Counting only attacks read as benign, baseline context cuts them from 3 to 1. The effect reproduces; the aggregate metric does not, and we say so.

E. Prompt-injection resilience

Against the full stack, no attack reaches benign on either arm in any repetition. The tier-3 NIM ensemble pairs Llama Guard 4 and the Nemotron safety guard; their coverage is complementary, blocking 6 of 10 payloads before the classifier runs. No single layer carries the safety claim: the sanitiser removes most of the surface, the fact overrides (read from the pod specification, not the attacker's bytes) neutralise physical implications, and the guards block residuals. A hostile command line cannot quarantine a healthy pod or suppress an attack on its own evidence; deterministic code is the last authority.

TABLE IV
Live paired A/B: baseline context on versus off, same incidents, live models. n = paired incidents; attacks → benign is the count of genuine attacks classified as benign.

Arm	Macro-F1	ΔF1	95% CI	McNemar p	n
gpt-oss-20b	0.7151	−0.0747	[−0.310, +0.144]	1.000	28
qwen3.8-27b	0.4802	−0.0026	[−0.178, +0.230]	0.344	28

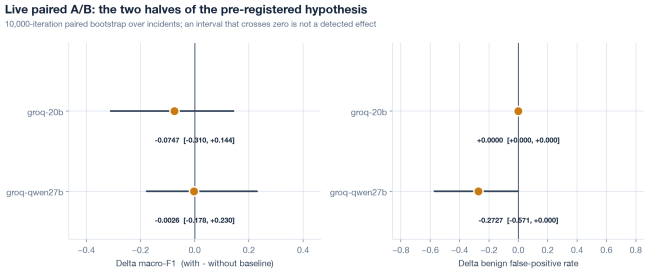


Fig. 4. Live paired A/B. Left: Δmacro-F1 (with minus without baseline context). Right: Δbenign false-positive rate. Both panels: paired bootstrap 95% intervals over 28 incidents.

F. Performance

A $5 \times 1{,}000$-event benchmark records 7,879.6 events/s with full hash-chained audit, $p_{50} = 122.6$ μs end-to-end, 0.00% errors. Normalisation is 33.6 μs, rule classification 1.9 μs, policy 1.5 μs and the tamper-evident audit write 85.9 μs (70% of budget)—the deliberate cost of verifiability; in-memory processing without the chain reaches 12,847.1 events/s.

VI. Discussion and Limitations

What is new. No verified system in the 2022–2026 literature combines all five: eBPF telemetry, an operator-declared baseline, one correlated window, a typed-decision LLM over runtime events, and a hash-chained audit trail. Closest precedents differ on one axis each: Shadowkube (learned baseline, honeypots, no typed decision); K8NTEXT (audit-log context, no eBPF/flows); GenKubeSec (configuration, not runtime); OCR-APT (narrative, not classification, no audit hash chain); GCID (formal evidence that rule engines cannot express multi-step correlation). Our contribution is the closed loop with a measured, pre-registered evaluation.

Threats to validity. (i) Lab, not production—all runs are controlled, benign-intent simulations; (ii) the live replication uses incident states derived from our own catalogue, not captured traffic; (iii) the test split is small (112 runs over 5 classes), intervals are wide, and the Macro-F1 effect is modest; (iv) hosted models are not deterministic at temperature zero—live numbers are averaged over repetitions and reported with ranges; (v) vendor accuracy figures are marketing-grade, not harness-verified; (vi) schema validity is not correctness—only the

TABLE V
Attack success rate by defence tier, live models (ASR, lower is better).

Arm	T0 raw	T1 sanitize	T2 overrides	T3 guard
gpt-oss-20b	20% [20-20]	0%	5% [0-10]	0%
qwen3.8-27b	10% [10-10]	0%	0%	0%

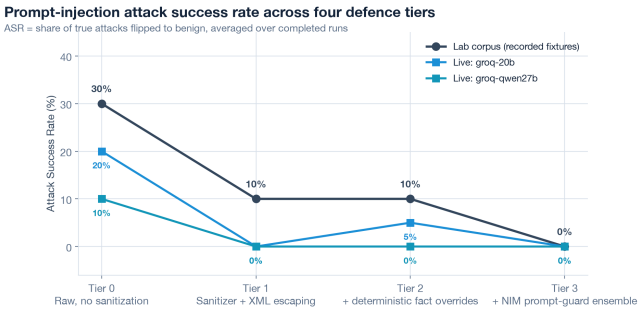


Fig. 5. Attack success rate by defence tier, lab fixtures and live models (mean over repetitions). ASR = share of true attacks flipped to benign.

fact overrides are guaranteed; (vii) eBPF tracing requires a standard Linux kernel.

VII. Conclusion

RuntimeLens conditions runtime threat detection on the one thing rule engines lack: an operator declaration of what each workload is supposed to do. Fusing five sources and classifying against that declaration yields a typed-decision, fully audited, SRE-safe response—3.6% benign false-positive rate, 94.7% containment precision, 0.0% prompt-injection success against the full defence stack. The honest headline is a real but modest Macro-F1 gain plus a substantial, systematic reduction in benign alerts; the live replication shows the same direction of effect on false positives while the aggregate metric remains inconclusive at n=28. Both are stated precisely. Future work: on-node quantised SLMs so telemetry never leaves the pod, and a field trial on a non-lab cluster.

Reproducibility

The evaluation harness, the 28-incident suite, raw per-call responses, figure scripts, deck generator and paper source are released with the repository (research/, docs/paper/). Re-running the A/B: uv run python -m research.live_typed_decision_eval --arms all.

References

[1] A. Asy Syairozi and Arizal, "Comparative analysis of eBPF-based runtime security monitoring tools," in RITECH, SCITEPRESS, 2025, pp. 136–141, doi:10.5220/0014272700004928.

[2] L. Galletta, "Enforcement of in-kernel stateful security policies via eBPF," arXiv:2609.13930, 2026.

[3] A. Islam and T. Rashid, "GCID: Graph-correlated intrusion detection," 2026, doi:10.2139/ssrn.7079427.

[4] J. Ryu et al., "Hybrid runtime detection of malicious containers using eBPF," Comput. Mater. Continua, vol. 86, no. 3, 2026, doi:10.32604/cmc.2025.074871.

[5] R. Kim et al., "Detecting cryptojacking containers using eBPF and ML," Electronics, vol. 14, no. 6, 2025, art. 1208, doi:10.3390/electronics14061208.

[6] Q. Chen et al., "Shadowkube," Cybersecurity, 2025, doi:10.1186/s42400-025-00372-7.

[7] X. Zou et al., "Towards generalizable context-aware anomaly detection," arXiv:2508.01844, 2025.

[8] I. Erakovic and C. Pahl, "Anomaly detection for partially observable container systems," in Proc. CLOSER, SCITEPRESS, 2025, doi:10.5220/0013224900003950.

[9] M. Franzil et al., "Sharpening Kubernetes audit logs with context awareness," Comput. Networks, vol. 276, 2026, doi:10.1016/j.comnet.2025.111890.

[10] A. Aly, E. Mansour, and A. Youssef, "OCR-APT," in Proc. ACM CCS, 2025, doi:10.1145/3719027.3765219.

[11] A. Oppong-Asante et al., "Runtime detection of attacks and misconfigurations in cloud-native Kubernetes using eBPF network telemetry," J. Comput. Virol., 2026, doi:10.1007/s11416-026-00671-8.

[12] E. Malul et al., "GenKubeSec: LLM-based Kubernetes misconfiguration detection, localization, reasoning, and remediation," arXiv:2405.19954, 2024.

[13] D. Sun, J. Zhang, Y. Tian, and Z. Li, "HIDBench," arXiv:2605.21773, 2026.

[14] S. A. Tsegai, A. Kantchelian, D. Yao, and P. Gao, "From investigation failures to reliable SOC agents," arXiv:2610.10608, 2026.

[15] R. Karanjai et al., "Context contamination in LLM analysis of network security logs," arXiv:2607.14493, 2026.

[16] H. Chen, Q. Long, and Y. Wang, "REFINE," arXiv:2609.32516, 2026.

[17] C. V. Garware and S. N. Zisad, "When the ruler is broken," arXiv:2605.07293, 2026.

[18] J. Ray, "The constraint tax," arXiv:2605.26128, 2026.

[19] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on tabular data?," in Proc. NeurIPS D&B, 2022.

[20] N. Pavlidis et al., "LLMs as universal predictors?," arXiv:2508.17391, 2025.

[21] I. H. Polas, "Tabular foundation models vs tuned gradient boosting," arXiv:2611.12942, 2026.

[22] A. Chakravorty and N. Elsayed, "Empirical evaluation of machine learning for cloud cryptojacking detection," arXiv:2610.20658, 2026.

[23] S. Ndichu et al., "AI-driven security alert screening and alert fatigue in security operations centers: A survey," arXiv:2605.08316, 2026.

[24] ScaleDown AI, "ScaleDown Platform Documentation and Typed Decisions API Reference," docs.scaledown.ai, 2026.

[25] Q. McNemar, "Note on the sampling error of the difference between correlated proportions," Psychometrika, vol. 12, no. 2, 1947, pp. 153–157.

[26] B. Efron and R. Tibshirani, An Introduction to the Bootstrap. New York, NY, USA: Chapman & Hall, 1993.

[27] M. W. Fagerland, S. Lydersen, and P. Laake, "The McNemar test for binary matched-pairs data," BMC Med. Res. Methodol., vol. 13, art. 91, 2013, doi:10.1186/1471-2288-13-91.

[28] C. A. Field and A. H. Welsh, "Bootstrapping clustered data," J. R. Stat. Soc. Ser. B, vol. 69, no. 3, pp. 369–390, 2007, doi:10.1111/j.1467-9868.2007.00593.x.

[29] J. Opitz and S. Burst, "Macro F1 and Macro F1," arXiv:1911.03347, 2019.

[30] S. Holm, "A simple sequentially rejective multiple test procedure," Scand. J. Stat., vol. 6, no. 2, 1979, pp. 65–70.